



# Stellar Bootcamp

Stellar & Soroban Bootcamp

Prof. Josiel Maimone de Figueiredo

Maio/2026

# Our Bootcamp Classes

**Class 1** (05/05) — Stellar Fundamentals

**Class 2** (12/05) — Custom tokens and Claimable Balances

**Class 3** (19/05) — Smart Contracts with Soroban

**Class 4** (26/05) — Integration and Final dApp

# Our Bootcamp Classes

**Class 1** (05/05) — Stellar Fundamentals

REVIEW

# Class 1 – Stellar Fundamentals

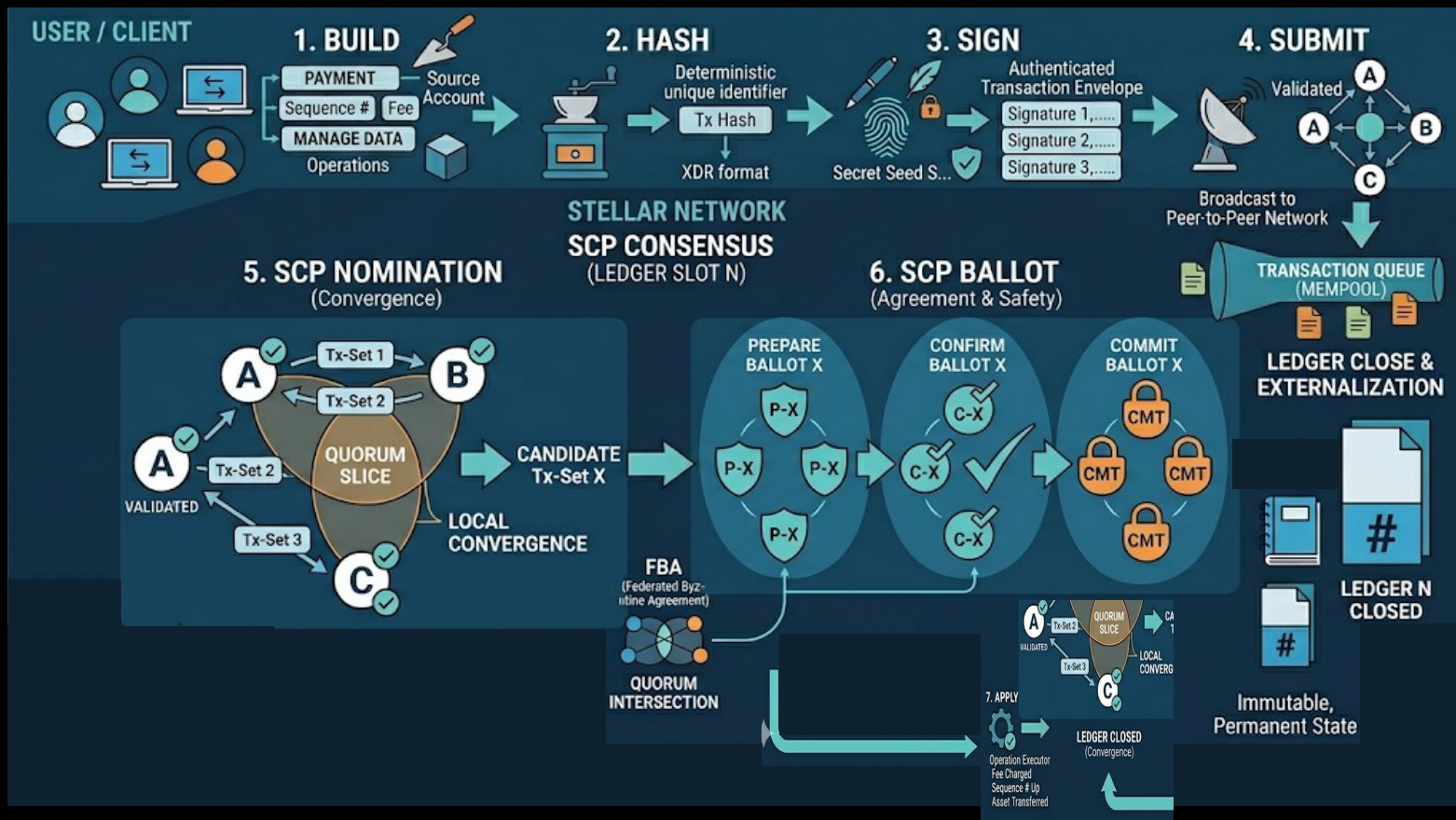
## Your “first” step on Stellar

- Purpose and vision of the Stellar blockchain.
- Differences between Stellar and other blockchains.
- Accounts, public keys, and private keys.
- Testnet and Mainnet.
- Environment setup with Stellar.

## Transactions and security

- Transaction structure on Stellar.
- Cryptographic signatures and account control.
- Simulation of Soroban transactions.
- Complete flow: construction, signing, simulation, and submission

# Stellar Fundamentals



# Class 2 – Stellar Fundamentals

Before coffee:

## Baguncinha journey

- Baguncinha transactions
- Stellar RPC journey

After coffee:

## Baguncinha token

- Smart token
- Claimable clawback

# Stellar RPC



## Jão's Baguncinha

“A complete story-driven tour  
of every Stellar RPC method  
via [lab.stellar.org](https://lab.stellar.org)”

# Stellar RPC



## Jão's Baguncinha

### Prologue:

- **X-Bagunça** is a kind of X-Tudo sandwich that exists on Cuiabá city.
- **Baguncinha** is a small X-Bagunça.
- Jão has as 15 years baguncinha's snack bar.





# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

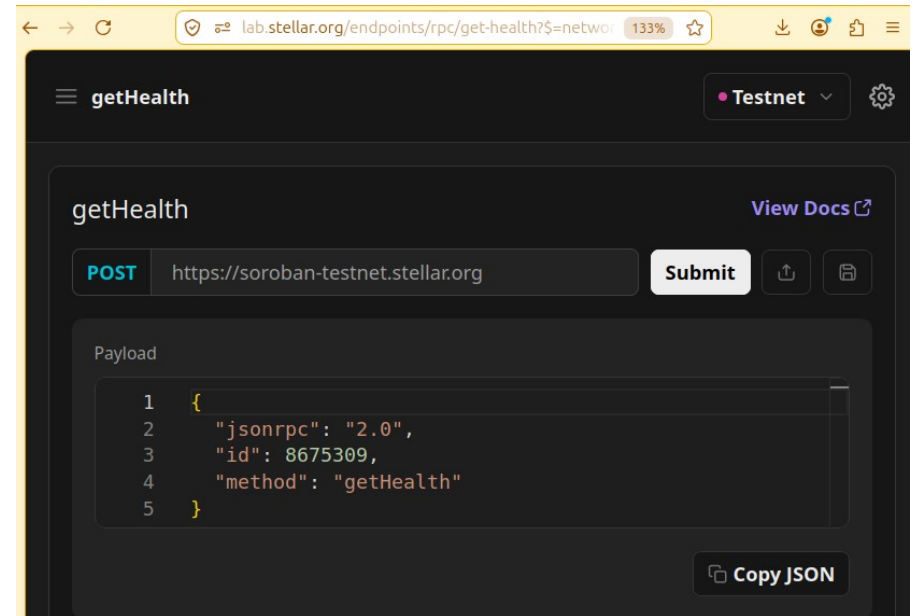
- 1) Check if the RPC is alive
- 2) Confirm the right network
- 3) Check version and supported protocol range
- 4) Get the current ledger
- 5) Browse the ledger history — Jão reviews
- 6) Simulate before charging yesterday's sales
- 7) Place the real order
- 8) Confirm payment
- 9) Generate day report
- 10) Check how many has left
- 11) Subscribe to events
- 12) Optimize fees

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 1) Check if the RPC is alive – before the first baguncinha
  - Monday morning, 7:00 AM. Jão arrives at his Baguncinha.
  - Before firing up the grill, he checks that his point-of-sale system (the Stellar RPC) is actually running. There's no point taking orders if the backend is down.
  - The **getHealth** method is the simplest RPC call — it returns whether the node is healthy and how many ledgers behind it is.
  - Jão runs this check every morning before accepting any payments.



# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

1) Javascript code for **getHealth**:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getHealth" }

let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json', },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

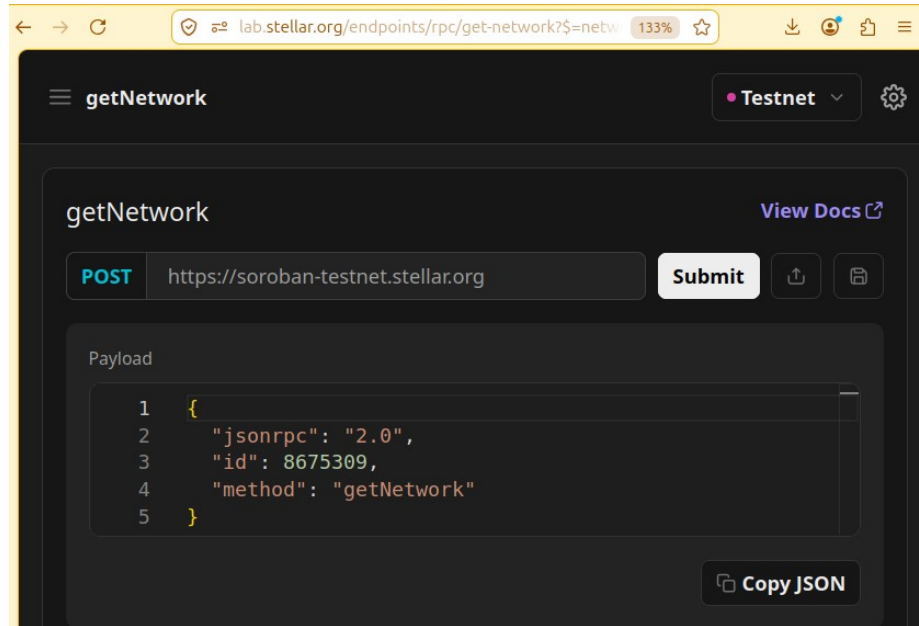
```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    status: 'healthy',
    latestLedger: 2505462,
    oldestLedger: 2384503,
    ledgerRetentionWindow: 120960
  }
}
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 2) Confirm we're on the right network — testnet or mainnet?
  - Jão realizes he has two point-of-sale tablets: one configured for training mode (testnet) and one for real money (mainnet). Before charging customers, he must confirm which tablet he's using.
  - **getNetwork** returns the network passphrase and protocol version.
  - The passphrase is cryptographically important — it's included in every transaction hash, making testnet signatures invalid on mainnet.



# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

2) Javascript code for `getNetwork`:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getNetwork" }

let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json', },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {

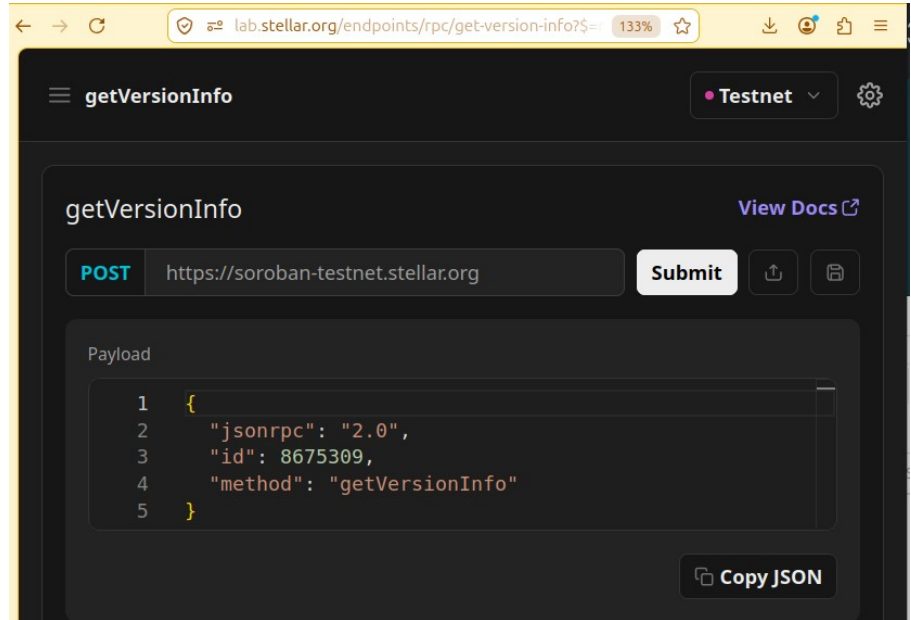
friendbotUrl: 'https://friendbot.stellar.org/',
    passphrase: 'Test SDF Network ; September
2015',
    protocolVersion: 26
  }
}
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 3) Check RPC software version and supported protocol range
- Jão's tech-savvy nephew told him to always log the software version before processing payments.
  - This helps when filing bug reports and confirming that the RPC supports the protocol version used by Jão's baguncinha contract.
  - **getVersionInfo** returns the stellar-core version, soroban-rpc version, commit hash, and the range of protocol versions supported.



# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

3) Javascript code for getVersionInfo:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getVersionInfo" }

let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json', },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

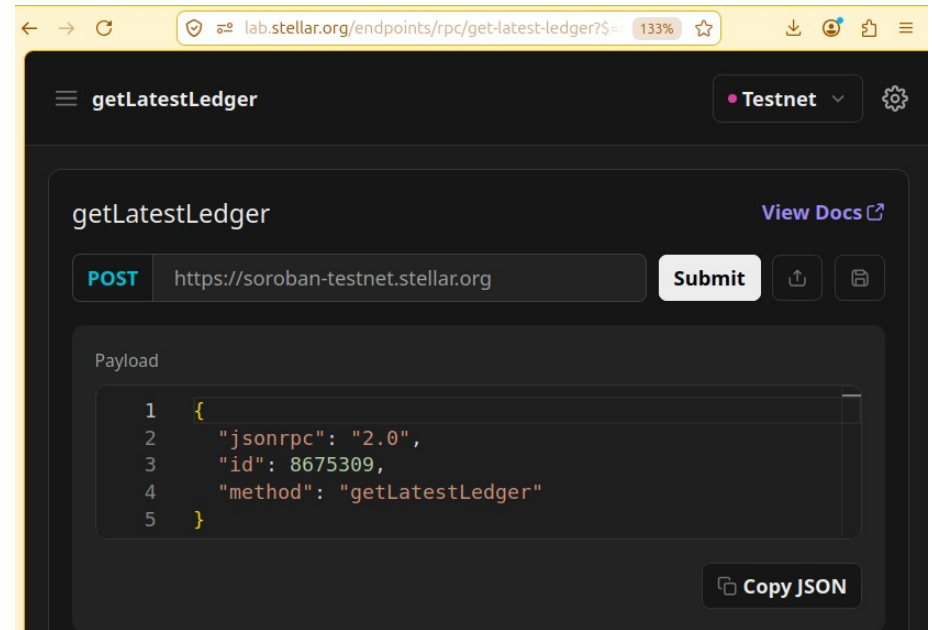
```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    version: '26.0.0-
b626d30b0541158335c6c8f060b75fcab671b8de',
    commitHash:
'b626d30b0541158335c6c8f060b75fcab671b8de',
    buildTimestamp: '2026-04-03T18:04:51',
    captiveCoreVersion: 'stellar-core 26.0.1
(e78c97ed0f517883cb7e93766e62f4b46b7d6d8b)',
    protocolVersion: 26
  }
}
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 4) Get the current ledger — the blockchain's heartbeat
- Every 5 seconds, a new ledger closes on Stellar. Jão thinks of each ledger as a "business day tick" — it's the timestamp of the blockchain.
  - He uses the latest ledger sequence to:
    - Set timeBounds on transactions (prevent stale payments)
    - Calculate TTL expirations for his baguncinha inventory
    - Confirm the RPC is synced to the live network





# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

4) Javascript code for `getLatestLedger`:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getLatestLedger" }
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',  },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

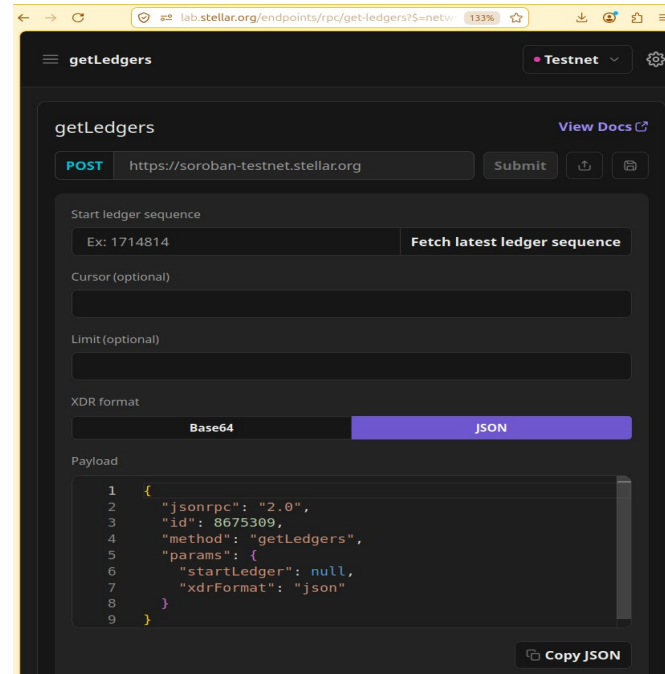
```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    id: 'f013...',
    protocolVersion: 26,
    sequence: 2508497,
    closeTime: '1778550992',
    headerXdr: 'AAAAGgT...',
    metadataXdr: 'AAAA...'
  }
}
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 5) Browse the ledger history — Jão reviews yesterday's sales
- At the end of the day, Jão wants to review the last 10 ledgers to audit all transactions processed at his stand.
  - **getLedgers** returns a paginated list of ledgers with their headers and metadata — like flipping through a receipt book.
  - He can see when each ledger closed, how many transactions it contained, and the cumulative base fee.
  - This is his "daily audit trail."



# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

5) Javascript code for getLedgers:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getLedgers" }

let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json', },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    Ledgers: [ {
      hash: '2c7...',
      sequence: 2507684,
      ledgerCloseTime: '1778546922',
      headerXdr: 'LHQ...',
      metadataXdr: 'AAAA...' } ],
    latestLedger: 2507686,
    latestLedgerCloseTime: 1778546932,
    oldestLedger: 2386727,
    oldestLedgerCloseTime: 1777941110,
    Cursor: '2507685' } }
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 6) Simulate before charging — dry run of the order contract
- A customer orders a "Classic Jão Special" (baguncinha + extra mustard).
  - Before charging their wallet, Jão's system runs a simulation — like doing a test swipe before the real transaction.
  - **simulateTransaction** is mandatory before any Soroban contract call.
  - It returns the exact fee to charge, the resource footprint (which storage entries will be read/written), and confirms the call won't fail.
  - Without it, the real transaction would be rejected.

The screenshot shows a web browser window with the URL `lab.stellar.org/endpoints/rpc/simulate-transaction`. The page title is `simulateTransaction` and it has a `Testnet` dropdown menu. Below the title, there is a `POST` button and a text input field containing `https://soroban-testnet.stellar.org`, followed by a `Submit` button and two icons (upload and download). The main content area is titled `simulateTransaction` and includes a `View Docs` link. It contains several input fields: `Transaction envelope XDR` with an example value, `Resource config - instruction leeway (optional)` with an example value of `1714814`, `Auth mode (optional)` with a dropdown menu set to `none`, and `XDR format` with two buttons: `Base64` and `JSON`.

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

6) Javascript code for simulateTransaction:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "simulateTransaction",
  "params": {
    "transaction": "AAAAA...AAA=",
    "resourceConfig": {
      "instructionLeeway": 3000000 } }}
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json', },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

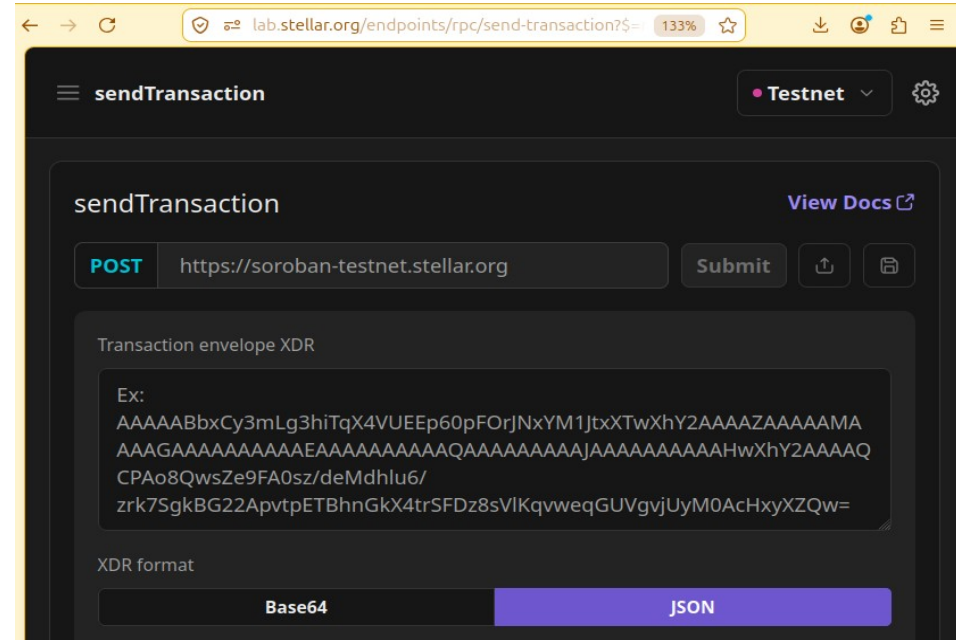
```
{
  "jsonrpc": "2.0",
  "id": 8675309,
  "result": {
    "transactionData": "AAAAA...==",
    "minResourceFee": "90353",
    "events": [
      "AAAA...=",
      "AAAA...="
    ],
    "results": [
      {
        "auth": [],
        "xdr": "AAAAAwAAAAw="
      }
    ],
    "cost": {
      "cpuInsns": "1635562",
      "memBytes": "1295756"
    },
    "latestLedger": 2552139 } }
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 7) Place the real order — customer pays with BAGUNCINHA tokens
  - The simulation passed! Now it's real.
  - The customer signs the transaction on their wallet and Jão's system submits it via **sendTransaction**.
  - This method returns immediately with a PENDING status and a transaction hash — the actual confirmation comes later via `getTransaction`.
  - Think of it as the customer handing over their card: the payment is initiated but not yet confirmed.



# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

7) Javascript code for sendTransaction:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "sendTransaction",
  "params": {
    "transaction": "AAAAA...w0="  }  }
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',  },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

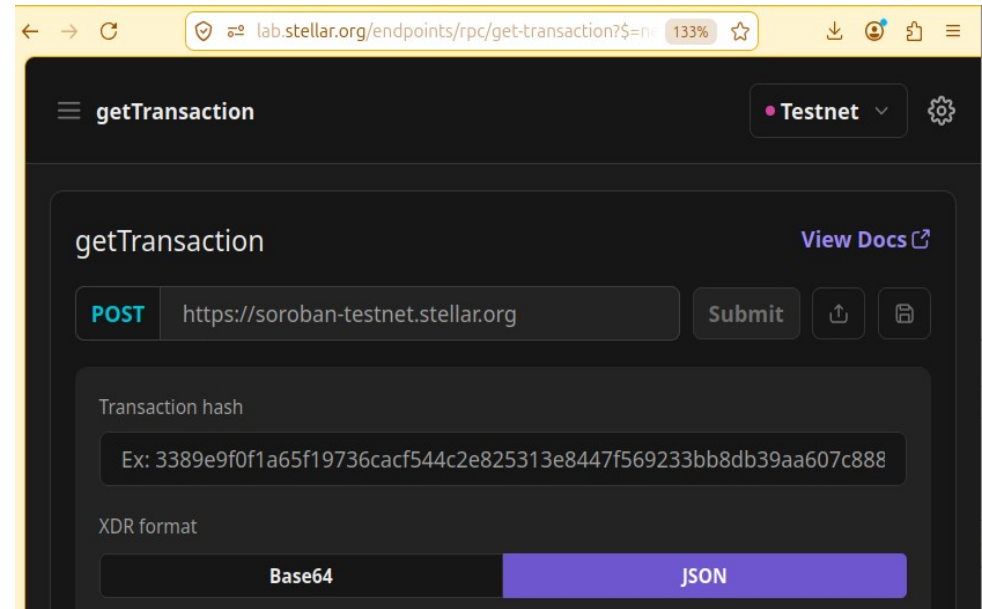
```
{
  "jsonrpc": "2.0",
  "id": 8675309,
  "result": {
    "status": "PENDING",
    "hash":
      "9dfc01af35a15909665dd1fed94a3bace3ebe90f30b4a78
      4025dcdb6b3188cc0",
    "latestLedger": 2513415,
    "latestLedgerCloseTime": "1778575612"  } }
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 8) Confirm the payment landed — print the receipt
- 5 seconds later. Jão polls **getTransaction** with the hash from the previous step.
  - It comes back SUCCESS. The baguncinha is paid for!
  - The response includes the full transaction envelope (what was submitted), the result XDR (what each operation returned), and all ledger entry changes (the side effects).
  - Jão's system parses the result to extract the order ID from the return value.





# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

8) Javascript code for getTransaction:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getTransaction",
  "params": {
    "hash": "32f7...7"  } }
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',  },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    latestLedger: 2513462,
    latestLedgerCloseTime: '1778575847',
    oldestLedger: 2392503,
    oldestLedgerCloseTime: '1777970038',
    status: 'SUCCESS',
    txHash: '9df...cc0',
    applicationOrder: 1,
    feeBump: false,
    envelopeXdr: 'AAA...AA=',
    resultMetaXdr: 'AAAA...AAA',
    events: {
      transactionEventsXdr: [
        'AAAA...AABk'  ],
      ContractEventsXdr: [ [
        'AAAA...+EdYA'  ]  ] },
    ledger: 2513417,
    CreatedAt: '1778575622' } }
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 9) End of day report — list all orders in ledger range
- It's 5 PM. Jão wants to generate a sales report for today.
  - He queries all transactions from ledger X to Y — the full business day.
  - **getTransactions** is the bulk version of `getTransaction`.
  - It returns paginated transaction history for a ledger range, perfect for building dashboards, analytics, or syncing an off-chain database.

A screenshot of a web browser showing the Stellar RPC interface for the `getTransactions` endpoint. The browser address bar shows `lab.stellar.org/endpoints/rpc/get-transactions?...`. The interface has a dark theme. At the top, it says `getTransactions` and `Testnet`. Below that, there's a `POST` method indicator and a text input field containing `https://soroban-testnet.stellar.org`, with a `Submit` button. Further down, there's a section for `Start ledger sequence` with an input field showing `Ex: 1714814` and a `Fetch latest ledger sequence` button. Below that are optional fields for `Cursor (optional)` and `Limit (optional)`. At the bottom, there's an `XDR format` section with two buttons: `Base64` and `JSON`, where `JSON` is currently selected. A `Payload` section is partially visible at the very bottom.

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

9) Javascript code for getTransaction:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getTransactions",
  "params": {
    "startLedger": 337272,
    "pagination": {
      "limit": 5    }  } }
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',  },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

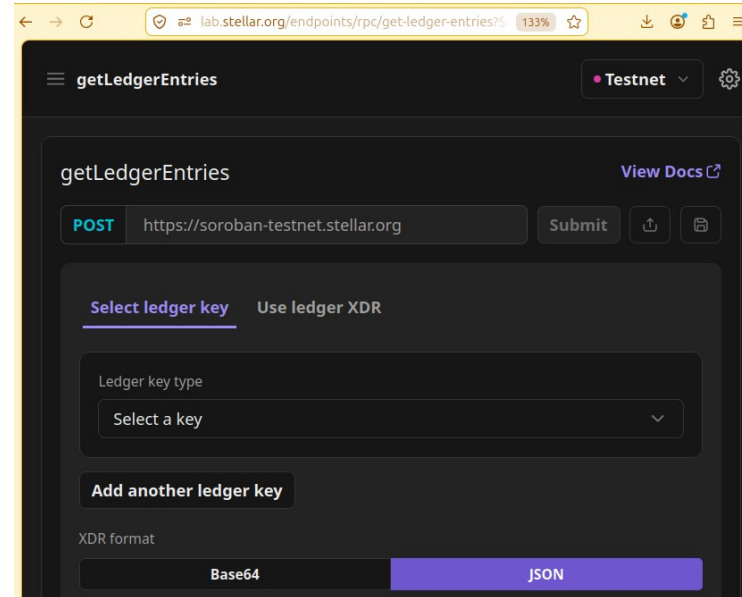
```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    transactions: [
      { status: 'SUCCESS',
        txHash: '9b9...78ae80',
        applicationOrder: 1,
        feeBump: true,
        envelopeXdr: 'AAAA...gg=',
        resultXdr: 'AAAA...4QAAAA',
        events: {
          transactionEventsXdr: ['AAAA...AOE'  ],
          contractEventsXdr: [...]  },
        ledger: 2513504,
        createdAt: 1778576057  },
      { status: 'FAILED',
        txHash: '43d...55c',
        ...
        createdAt: 1778576057  },
      ... ],
    latestLedger: 2513505,
    latestLedgerCloseTimestamp: 1778576062,
    oldestLedger: 2392546,
    oldestLedgerCloseTimestamp: 1777970254,
    cursor: '10795417478385665'  }}
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 10) Check live contract state — how many baguncinha are left?
- A bus just arrived and 30 tourists want baguncinhas.
  - Before Jão starts cooking, he checks his inventory to see how many are in stock.
  - **getLedgerEntries** reads the live on-chain state directly — no need to replay events.
  - Jão queries the inventory key in his baguncinha persistent storage.
  - This is the most efficient way to read contract data without invoking a transaction.



# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

10) Javascript code for getLedgerEntries:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getLedgerEntries",
  "params": {
    "keys": [
      "AAAAABgAAAAHMA/50/Q+w3Ni8UXWm/trxFBfAfl6De5kFttaMT0/ACwAAA
      BAAAAABAAAAA8AAAAHQ291bnRlcgAAAAASAAAAAAAAAAAAAg4dbAxsG
      AGICfBG3iT2cKGYQ6hK4sJWzZ6orlC5v6GAAAAAE=" ] } }
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json', },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    entries: [],
    latestLedger: 2513717 }
}
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

- 11) Subscribe to contract events — alerts for Jão's stand
  - Jão's nephew built a dashboard that listens to events emitted by the baguncinha contract. Whenever an order is placed, the contract emits a `order_placed` event.
  - Whenever inventory drops below 10, it emits a `low_stock` event.
  - **getEvents** lets you search for events by contract ID, topic filters, and ledger range.
  - It's the primary way to build real-time dashboards and off-chain indexers for Soroban contracts.

A screenshot of a web browser showing the Stellar RPC interface for the `getEvents` endpoint. The browser address bar shows `lab.stellar.org/endpoints/rpc/get-events?network=Testnet`. The interface has a dark theme. At the top, it says "getEvents" and "Testnet". Below that, there's a "POST" method selector and a text input field containing `https://soroban-testnet.stellar.org`, with a "Submit" button. The main form area has several sections: "Start ledger sequence" with an input field containing "Ex: 1714814" and a "Fetch latest ledger sequence" button; "End ledger sequence (optional)" with an input field containing "Ex: 1714814"; "Cursor (optional)" with an empty input field; "Limit (optional)" with an empty input field; "Filters type" with a dropdown menu showing "Select filters type"; and "XDR format" with two buttons, "Base64" and "JSON", where "JSON" is highlighted in blue.

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

11) Javascript code for getTransaction:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getTransaction",
  "params": {
    "hash":
"32f7e5c3afd281fcaa99c0e990adf62f33e3bb341b1641a5c8b0b4a4d
c55c487"  }}
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',  },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    latestLedger: 2513898,
    latestLedgerCloseTime: '1778578030',
    oldestLedger: 2392939,
    oldestLedgerCloseTime: '1777972222',
    status: 'SUCCESS',
    txHash: '9dfc0...3188cc0',
    applicationOrder: 1,
    feeBump: false,
    envelopeXdr: 'AAAA...K5IwsviIw4=',
    resultXdr: 'AAAA...AAAAA=',
    resultMetaXdr: 'AAAA...QAAAAA',
    events: {
      transactionEventsXdr: [ 'AAAA...Bk'  ],
      contractEventsXdr: [ [ 'AAA...EdYA'  ]  ]  },
    ledger: 2513417,
    createdAt: '1778575622'  }  }
```

# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

12) Optimize fees — Jão checks what everyone else is paying

- During the lunch rush, many customers are submitting transactions simultaneously.
- Jão's system uses **getFeeStats** to check current fee market conditions and set an optimal fee that will get included in the next ledger without overpaying.
- This returns percentile fee data for both classic and Soroban transactions across recent ledgers — essentially a fee gas price oracle.

The screenshot shows a web browser window with the URL `lab.stellar.org/endpoints/rpc/get-fee-stats?network=Testnet`. The page title is `getFeeStats` and it includes a `Testnet` dropdown menu. The main section is titled `getFeeStats` with a `View Docs` link. Below this, there is a `POST` method indicator and a text input field containing `https://soroban-testnet.stellar.org`, followed by a `Submit` button. A `Payload` section contains a JSON snippet:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 8675309,
4   "method": "getFeeStats"
5 }
```

At the bottom right of the payload section is a `Copy JSON` button.



# Stellar RPC



Jão uses RPC Stellar in his daily financial activities:

12) Javascript code for **getFeeStats**:

```
let requestBody = {
  "jsonrpc": "2.0",
  "id": 8675309,
  "method": "getFeeStats" }
let res = await fetch('https://soroban-
testnet.stellar.org', {
  method: 'POST',
  headers: {'Content-Type':
'application/json',
  },
  body: JSON.stringify(requestBody), })
let json = await res.json()
console.log(json)
```

Answer:

```
{
  jsonrpc: '2.0',
  id: 8675309,
  result: {
    sorobanInclusionFee: {
      Max: '200', min: '100',
      Mode: '100', P10: '100', p20: '100',
      P30: '100', p40: '100', P50: '100',
      P60: '100', P70: '100', p80: '100',
      P90: '100', p95: '100', P99: '200',
      transactionCount: '243',
      LedgerCount: 50    },
    inclusionFee: {
      Max: '200', min: '100',
      Mode: '100', p10: '100', P20: '100', p30: '100',
      P40: '100', p50: '100', P60: '100', p70: '100',
      P80: '100', p90: '200', P95: '200', p99: '200',
      transactionCount: '19',
      LedgerCount: 10    },
    LatestLedger: 2508681  }  }
```

# Stellar RPC



## Jão's Baguncinha

Evolution:

- Jão launches **BAGU** — his Baguncinha loyalty token
- **Baguncinha** has its token!



# Stellar RPC



## Baguncinha Token: BAGU

- 1) João launches **BAGU** — his Baguncinha loyalty token
  - customers earn BAGU asset with every purchase.
  - João needs two accounts:
    - **Issuer** — the mint account (kept offline, ultra-secure) for BAGU token.
    - **Distributor** — the daily operational wallet that rewards customers.
  - **Zé** — loyal customer account who will earn and spend BAGU.

### Let's create the accounts in Stellar Lab:

#### 1) Account → Create account keypair

##### 1) Generate keypair → Fund 10,000 XLM → Save keypair

- Issuer
- Distributor
- Zé account

# Stellar RPC



## Baguncinha Token: BAGU

2) Jão launches **BAGU** — his Baguncinha loyalty token

- Jão configures clawback on the BAGU issuer
- Before any tokens exist, Jão sets compliance flags on the Issuer.
- He anticipates fraud from day one. Two flags are needed:
  - `AUTH_REVOCABLE` — prerequisite, allows revoking trustline auth.
  - `AUTH_CLAWBACK_ENABLED` — allows recovering tokens from any account.
- Every trustline created after this moment will inherit `TRUSTLINE_CLAWBACK_ENABLED_FLAG` automatically.

Let's operate in Stellar Lab:

1) Build transaction

- 1) Source account → <Issuer account>
- 2) Fetch next sequence
- 3) Operation 0 → Operation Type
  - **Set Options** → **Authorization revocable** and **Authorization clawback enabled**
- 4) Sign transaction
- 5) Use secret key → Sign → Submit Transaction
- 6) Submit Transaction

–

# Stellar RPC



## Baguncinha Token: BAGU

- 3) BAGU tokens are mint and ready for distribution
- Distributor open trust for BAGU receivement

Let's mint BAGU and send to Distributor:

- Build transaction
  - Source account → <Distributor account>
  - Fetch next sequence
  - Operation 0 → Operation Type
    - **Change Trust**
    - **Asset** → **Alphanumeric 4**
    - **Asset code** → **BAGU**
    - **Issuer account ID** → <Issuer account>
  - Sign transaction → Use secret key → Sign → Submit Transaction
  - Submit Transaction

# Stellar RPC



## Baguncinha Token: BAGU

- 3) BAGU tokens are mint and ready for distribution
- Distributor open trust for BAGU receivment
  - Issuer mints BAGU and send to Distributor
    - Payment → BAGU → Distributor
  - Confirm on:

<https://stellar.expert/explorer/testnet/account/>

Let's mint BAGU and send to Distributor:

- Build transaction
  - Source account → <Issuer account>
  - Fetch next sequence
  - Operation 0 → Operation Type
    - **Payment**
    - **Destination** → <Distributor account>
    - **Asset** → Alphanumeric 4
    - **Asset code** → BAGU
    - **Amount** → 100000
  - Sign transaction → Use secret key → Sign → Submit Transaction
  - Submit Transaction

# Stellar RPC



## Baguncinha Token: BAGU

### 4) Zé should change trust to receive BAGU

- With flags set, the Distributor and Zé each create a trustline with Issuer - an explicit opt-in to hold BAGU.
- Without it, no account can receive custom tokens. Each trustline adds +0.5 XLM to the account's minimum balance.
- Because AUTH\_REQUIRED is NOT set, the trustlines are immediately active.
- Both trustlines inherit TRUSTLINE\_CLAWBACK\_ENABLED\_FLAG = true automatically.

### Let's prepare Zé account to receive BAGU:

- Build transaction
  - Source account → <Zé account>
  - Fetch next sequence
  - Operation 0 → Operation Type
    - **Change Trust**
    - **Asset** → **Alphanumeric 4**
    - **Asset code** → **BAGU**
    - **Issuer account ID** → **<Issuer account>**
  - Sign transaction → Submit Transaction
- Submit Transaction

# Stellar RPC



## Baguncinha Token: BAGU

- 5) Zé buys a Baguncinha and earns 10 BAGU
- Zé orders the "Double Stack Special" for \$5.
  - João's POS system automatically triggers a reward: 10 BAGU sent from the Distributor to Zé.
  - The reward lands in Zé's wallet within 5 seconds.
  - Zé now has 10 BAGU to use on his next visit.
  - The loyalty program is working!

Let's pay BAGU to Zé:

- Build transaction
  - Source account → **<Distributor account>**
  - Fetch next sequence
  - Operation 0 → Operation Type
    - **Payment**
    - Destination → **<Zé account>**
    - Asset → Alphanumeric 4
    - Asset code → **BAGU**
    - Amount → **10**
  - Sign transaction → ... → Submit transaction



# Stellar RPC



## Baguncinha Token: BAGU

5) Jão creates a 50 BAGU weekend promo with time window predicate

- Jão wants to increase traffic.
- He creates a Claimable Balance of 50 BAGU — with a predicate: claimable after 5 min.
- A safety net second claimant is also added:
  - if Zé doesn't claim during the window, Jão's Distributor can recover the tokens after 7 min.
  - Without this, the balance would be stuck forever.

Let's create the promo:

- Build transaction
  - Source account → <Distributor account>
  - Fetch next sequence
  - Operation 0 → Operation Type
    - **Create Claimable Balance**
    - **Issuer account ID** → <Issuer account>
    - **Amount** → 50
    - **Add Claimant** → Destination → <Zé account>
      - Conditional → NOT → Conditional → Time → 300
    - **Add Claimant** → Destination → <Distributor account>
      - Conditional → NOT → Conditional → Time → 420
  - Sign transaction → ... → Submit Transaction

# Stellar RPC



## Baguncinha Token: BAGU

- 5) Jão creates a 50 BAGU weekend promo with time window predicate
- Jão wants to increase traffic.
  - He creates a Claimable Balance of 50 BAGU — with a predicate: claimable after 5 min.
  - A safety net second claimant is also added:
    - if Zé doesn't claim during the window, Jão's Distributor can recover the tokens after 7 min.
    - Without this, the balance would be stuck forever.

Let's make Zé claim its tokens (after 5 min):

- Build transaction
  - Source account → <Zé account> → Fetch next sequence
  - Operation Type → **Claim Claimable Balance**
    - **Claimable balance ID** → **<claimable ID>**
  - Sign transaction → ... → Submit Transaction
- **<claimable ID>** → Horizon endpoints → Claimable balances → All claimable balances

# Stellar RPC



## Baguncinha Token: BAGU

- 6) João discovers Carlos bought BAGU on the grey market
- João reviews analytics. Carlos has 500 BAGU but only bought two Baguncinhas — he should have at most 20 BAGU.
  - João checks the payment history and finds Carlos purchased tokens from other customers peer-to-peer. This violates the loyalty terms.
  - João's clawback powers are activated. Since Carlos created his trustline after the flags were set, `TRUSTLINE_CLAWBACK_ENABLED_FLAG = true` on his BAGU trustline.

Stop BAGU grey market:

- Build transaction
  - Source account → <Issuer account> → Fetch next sequence
  - Operation Type → **Clawback**
    - **Asset** → Alphanumeric 4 → **BAGU**
    - **From** → <Carlos account>
    - **Amount** → **480**
  - Sign transaction → ... → Submit Transaction

# Stellar RPC



## Baguncinha Token: BAGU

- 7) Atomic: freeze Carlos then clawback his remaining 20 BAGU
- Carlos complains loudly and João decides: full ban.
  - First, João freezes Carlos's BAGU trustline (so Carlos can't spend his last 20 BAGU before the second operation executes).
  - Then immediately claws back the remaining 20 BAGU.
  - Both operations are in a single atomic transaction. Either both succeed or neither does — no gap where Carlos could race-condition-spend his tokens.

Stop BAGU grey market and freeze Carlos:

- Build transaction
  - Source account → <Issuer account> → Fetch next sequence
  - Operation 0 → Operation Type → **Set Trust Line Flags**
    - **Asset** → Alphanumeric 4 → **BAGU**
    - **Trustor** → <Carlos account>
    - **Set flags** → **Authorized to maintain liabilities**
    - **Clear flags** → **Authorized**
  - Operation 1 → Operation Type → **Clawback**
    - **Asset** → Alphanumeric 4 → **BAGU**
    - **From** → <Carlos account>
    - **Amount** → 20
  - Sign transaction → ... → Submit Transaction

# Stellar RPC



## Baguncinha Token: BAGU

- 8) Jão cancels a pending claimable balance created by mistake
- Jão had created a 200 BAGU promotional balance for his nephew Jão Jr., but accidentally set the wrong amount (should have been 20, not 200).
  - Jão Jr. hasn't claimed it yet. Jão uses `ClawbackClaimableBalance` to cancel the entire pending balance.
  - This is different from the regular `ClawbackOp`:
    - it operates on the pending claimable balance entry in the ledger — not on a trustline.
    - And it always destroys the full amount — no partial option.

Let's make Zé claim its tokens (after 5 min):

- Build transaction
  - Source account → <Issuer account> → Fetch next sequence
  - Operation 0 → Operation Type → **Clawback Claimable Balance**
    - **Claimable Balance Id** → <claimable ID>
  - Sign transaction → ... → Submit Transaction

# Stellar RPC



## Baguncinha Token: BAGU

8) Jão locks the issuer — BAGU supply is fixed at 1,000,000 forever

- After 6 months, the program has distributed 800,000 BAGU.
- Jão decides to fix the supply permanently.
- Customers will know their tokens are scarce and not subject to inflation.
- He zeroes the masterWeight on the Issuer account.
- From this moment, no new BAGU can ever be minted.
- Anyone can verify this by checking masterWeight = 0 on the Issuer account.

Let's make BAGU token unmint:

- Build transaction
  - Source account → <Issuer account> → Fetch next sequence
  - Operation 0 → Operation Type
    - **Set Options**
      - **Master weight** → 0
  - Sign transaction → ... → Submit Transaction

# Stellar RPC



Jão's Baguncinha

“Our tour on BAGU economy is complete!!!”



# Stellar Bootcamp

Next class (19/5):  
**Smart Contracts with Soroban**  
(Rust, rust, rust,...)